

Log-linear models

Back to language modeling

Goal: assign $p(x_1, \dots, x_n)$ to every sentence

$$\text{s.t. } \sum_{\substack{x_1, \dots, x_n \\ \in V^*}} p(x_1, \dots, x_n) = 1 \quad \leftarrow \text{HW Q1 about this}$$

usually, modeled "autoregressively"

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$

Markovian assumption

$$p(x_i | x_1, \dots, x_{i-1}) = p(x_i | x_{i-2}, x_{i-1})$$

(or last k)

reviewer studied ways to average over various k -gram models

$$q(w/u, v) = \lambda_1 q_{\text{tri}}(w/u, v) + \lambda_2 q_{\text{bi}}(w/v) + \lambda_3 q_{\text{uni}}(w)$$

Q2 on
HW about
understanding
this

Idea what if you wanted ^{to} model

dist of w given more specific conditions:

no restricted / specific cond of u or v ...

eg restricted / specific - ... - bigram model

$$q_1(w_i | X[w_{i-1} \text{ is an adjective}])$$

intuition give high prob to $w_i = \text{nouns}$
big {house, meal}

next highest probs to $w_i = \text{adjectives}$

+ clever $p(\text{green} | \text{adj} = 1)$ big
big $\leftarrow$ green house

green big house $\leftarrow$ low prob to $p(\text{big} | \text{adj} = 1)$

big Italian meal

Italian big meal $\leftarrow$ unlikely
 $\swarrow$ part of speech

could work if we had a dictionary $\text{pos}(w)$

$\text{pos}(\text{"green"}) = \text{adj}$

$\text{pos}(\text{"dog"}) = \text{noun}$ etc.

eg "restricted condition" $\swarrow$ last 4 characters

$$q_2(w_i | X[\text{suff 4}(w_{i-1}) = \text{"ical"}])$$

list of w_i | prev word ends in ical

eg theoretical, magical

w_i likely noun, abstract noun "abstract"

eg $q_2(\text{"idea"} | X=1)$ high

could have long context conditions

$$q_3(w_i \mid X[w_j = \text{"cat"} \text{ for any } j \in \{1, \dots, i-1\}])$$

none of these examples are useful on their own,
but imagine

$$\begin{aligned} p(w_i \mid w_1, \dots, w_{i-1}) = & \lambda_1 q_1(w_i \mid \text{pos}(w_{i-1})) \\ & + \lambda_2 q_2(w_i \mid \text{suff}_4^X(w_{i-1}) = \text{"ical"}) \\ & + \lambda_3 q_3(w_i \mid \exists \text{"cat"} \text{ in } w_1, \dots, w_{i-1}) \leftarrow \\ & + \lambda_4 q_{\text{tri}}(w_i \mid w_{i-2}, w_{i-1}) + \lambda_5 q_{b_i} + \lambda_6 q_{u_i} \end{aligned}$$

becomes unwieldy

Today: make this work

Def Log-linear model is

- 1 X : set of inputs $\leftarrow$ can be infinite
 - 2 Y : set of output labels $\leftarrow$ finite
 - 3 d : # of features (Correspond to q_1, q_2 , etc in example)
 - 4 function $f: X \times Y \rightarrow \mathbb{R}^d$
- map each pair (x, y) to "feature vector" $f(x, y) \in \mathbb{R}^d$

map to any x, y

parameter / weights $v \in \mathbb{R}^d$

Define a cond. model of y given x :

$$p(y|x) = \frac{e^{v \cdot f(x,y)}}{\sum_{y' \in \mathcal{Y}} e^{v \cdot f(x,y')}} \quad \leftarrow \begin{aligned} &= \text{dot product} \\ &v \cdot f(x,y) \\ &= \sum_{k=1}^d v_k \cdot f_k(x,y) \end{aligned}$$

For language modeling, $X = V^*$ partial sentences
 $y = V$

from our example, log-linear model w/ same info

$$f((x_1, \dots, x_{i-1}), {}^y x_i) =$$

$$\left[\begin{aligned} f_1(x,y) &= \begin{cases} 1 & \text{if } x_{i-1} \text{ is adj} \\ 0 & \end{cases} \\ f_2(x,y) &= \begin{cases} 1 & \text{if suffix}(x_{i-1}) = "ical" \\ 0 & \end{cases} \\ f_3(x,y) &= \begin{cases} 1 & \text{if suffix}(x_{i-1}) = "able" \\ 0 & \end{cases} \\ \vdots \\ f_7(x,y) &= \begin{cases} 1 & \text{if } x_j = "cat" \text{ for } j < i \\ 0 & \end{cases} \\ f_8(x,y) &= \begin{cases} 1 & \text{if } {}^y x_i = "is" \text{ and } x_{i-2}, x_{i-1} = "the", "cat" \\ 0 & \end{cases} \\ \vdots \end{aligned} \right.$$

etc
 version of this $\forall$ trigrams x_{i-2}, x_{i-1}, x_i

- huge n
- in $\{0,1\}^d$ (often is) binary feature vector
- most features come from "template"
 - eg trigram features for each triplet of words
 - $|V|^3$ of those

represent the templated features in compact way
for (u, v, w) create/add feature

$$f_{N(u,v,w)}(x,y) = \begin{cases} 1 & \text{if } y=w, w_{i-2}, w_{i-1} = u, v \\ 0 & \end{cases}$$

$N(u,v,w) : V^3 \rightarrow \mathbb{N}$ is injective

(unique / distinct intge for each trigram)

similarly template for "suff 4" features

$S_4 = \{ \text{all } v \text{ st } v = \text{suff}_4(w) \text{ for any } w \text{ in data} \}$

for $v \in S_4$ add

$$f_{N(v, \odot)}(x,y) = \begin{cases} 1 & \text{if } y=w, \text{suff}(w_{i-1})=v \\ 0 & \end{cases}$$

introduces

$|V| \times |S_4|$

$\nearrow$ injective

d is huge

but to compute $p(y|x)$ need $v \cdot f(x,y)$ only

Idea: if $f(x,y)$ super sparse := mostly 0s

(and few 1s)

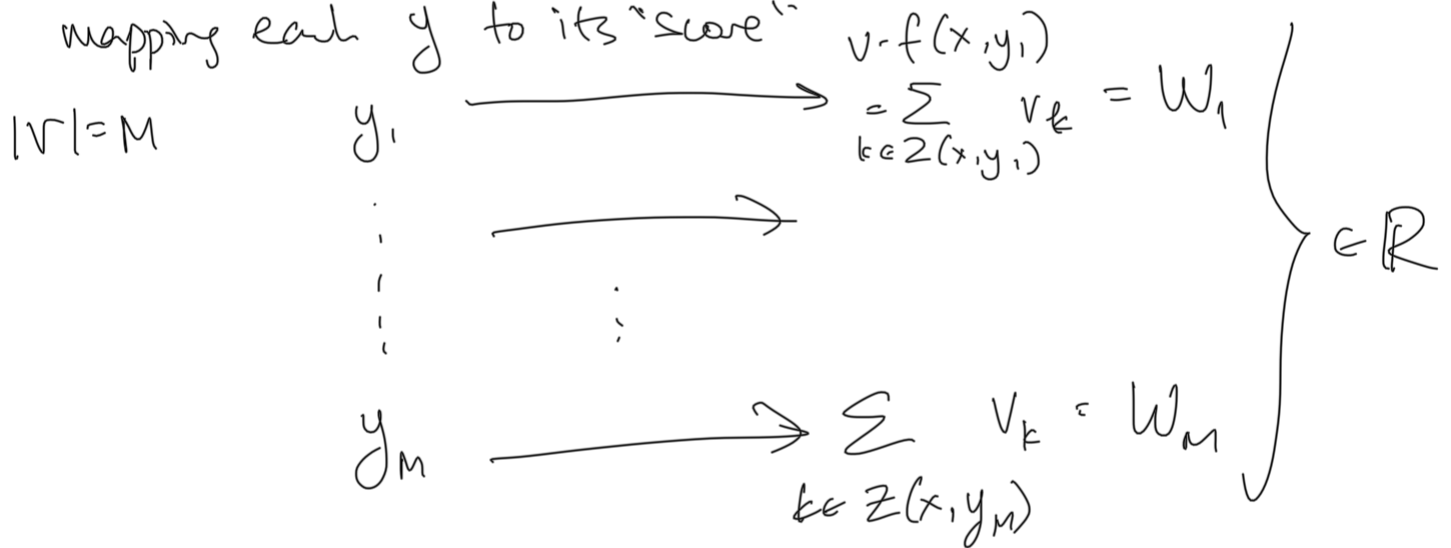
Assume can compute efficiently set

$$Z(x, y) = \{k : f_k(x, y) = 1\}$$

then $\underline{v \cdot f(x, y)} = \sum_{k \in Z(x, y)} v_k$ ✓

↳ necessary info from $f(x, y)$ 'just this

given fixed x , $\underline{v \cdot f(x, y)}$ doing
mapping each y to its "score"



maps each candidate y_i to score $W_i \in \mathbb{R}$

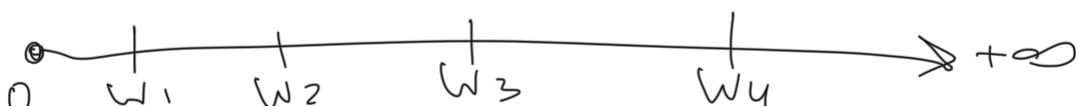
not probs.

want: big, positive $W_i \rightarrow$ give high probs

neg $W_i \rightarrow$ low prob

how to transform into probs?

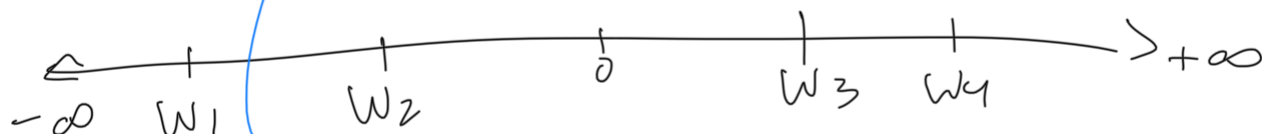
easy case



$\forall w_i \geq 0$ make into probs:

$$p_i = \frac{w_i}{w_1 + w_2 + w_3 + w_4}$$

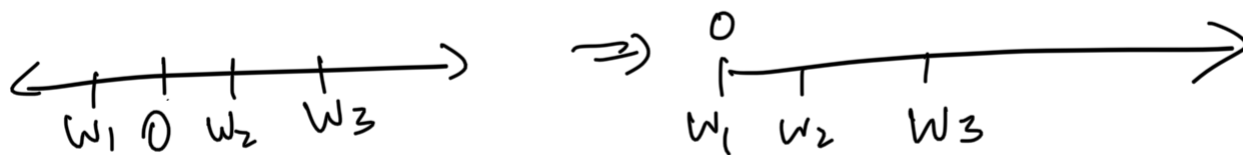
problem



doesn't give prob. dist over w_i

Idea: "shift up" by $|w'|$ $w' = \min_i w_i$

$$\forall j \quad w_j \mapsto w_j + |w'|$$



then use $p_i = \frac{w_i}{\sum w_j}$ ← modified

problem $p_3 = \frac{w_3 + |w'|}{w_2 + w_3 + 2|w'|}$

as $w_1 \rightarrow -\infty$, w_2 and w_3 fixed / as were,

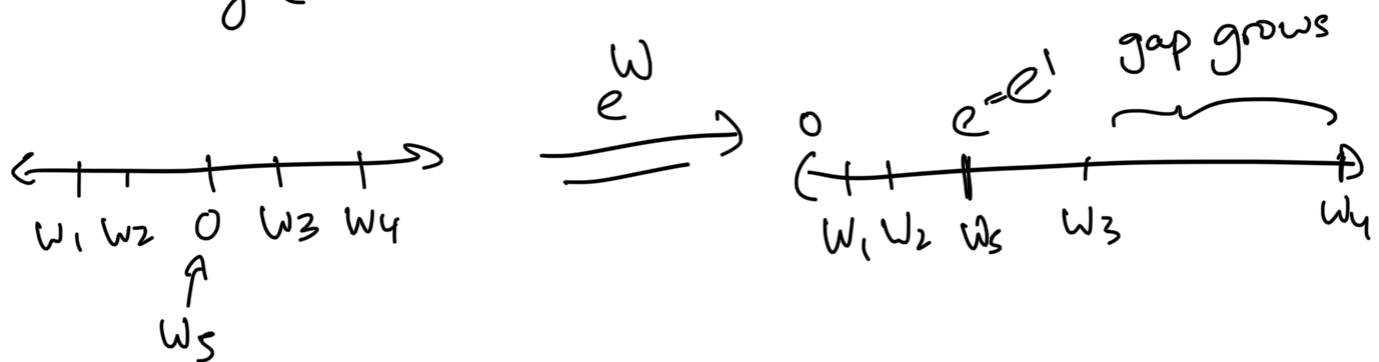
$p_2, p_3 \rightarrow \frac{1}{2}$ their relative gap erased

Idea w/o this problem

use function that's monotonic (order preserving)

$$\mathbb{R} \longrightarrow \mathbb{R}_{>0}$$

$$g(W) = e^W$$



blows up big W

but arbitrarily taking any weight $\min(W_i) \rightarrow -\infty$

doesn't have effect (goes to having 0 effect

as opposed to increasing effect in shift selection)

$$p_i = \frac{e^{W_i}}{\sum_{j \in Y} e^{W_j}}$$

this is called

$\text{softmax}(W_1, \dots, W_n)$

note people might say $\text{softmax}(w_i)$
but that's undefined, has to be
relative to $w_1, \dots, w_n$

origin of name "log linear models"

$$p(y|x) = \frac{e^{v \cdot f(x,y)}}{\sum_{y' \in Y} e^{v \cdot f(x,y')}}$$

$$y \in \mathcal{Y}$$

$$\log p(y|x) = \underbrace{v \cdot f(x, y)}_{\text{depends on } y} - \underbrace{\log \left(\sum_{y' \in \mathcal{Y}} e^{v \cdot f(x, y')} \right)}_{:= g(x)}$$

Constant wrt y

log prob of y = linear in the feature vector + constant

Parameter estimation

(f decided / hard-coded)

how to choose weights v ? Given data

$$\mathcal{C} = \{ (x^{(i)}, y^{(i)}) \}_{i=1}^m$$

might want

$$v_{ML} = \arg \max_{v \in \mathbb{R}^d} \sum \log p(y^{(i)} | x^{(i)})$$

parameter estimation

$$\text{MLE} \quad V_{\text{ML}} = \arg \max_{V \in \mathbb{R}^d} \sum_{i=1}^m \log p_v(y^{(i)} | x^{(i)})$$

problem w/ letting v be unconstrained

Suppose trigram template feature in f

"the big cat" occurs once in C

and that is the only instance of "the big"

$$f_N(\text{the, big, cat}) = f_N^{\sim} \quad \text{on } i=100, x^{(100)}, y^{(100)} \\ f_N^{\sim}((\dots, \text{the, big}), \text{cat}) = 1$$

then use MLE as above $V_N^{\sim} \rightarrow +\infty$

Pf $f_N^{\sim} = 1$ only on $x^{(100)}, y^{(100)}$ training example

$$p(\text{"cat"} | \dots \text{is the}) = e^{V_N^{\sim} \cdot 1 + \dots}$$

$$\begin{aligned} \text{"cat"} \rightarrow \sum_{(w)} e^{v \cdot f(x, w)} \\ \text{as } V_N^{\sim} \rightarrow \infty \quad &= \frac{e^{\infty + \dots}}{e^{\infty + \dots} + \sum_{w \neq \text{cat}} e^{w_2} + \dots + e^{w_{(v)}}} \end{aligned}$$

cat $\underbrace{\quad}_{\text{unchanged by}}$
 $f_{\tilde{N}}(x, w) = 0$

$\rightarrow 1$

Regularization

$$L(v) = \sum_{i=1}^m \log P_v(y^{(i)} | x^{(i)}) - \frac{\lambda}{2} \|v\|_2^2$$

$$\|v\|_2^2 = \sum_{i=1}^d v_i^2 \quad L_2$$

$$\|v\|_1 = \sum_{i=1}^d |v_i| \quad L_1$$

scaling
constant
 $\lambda > 0$

$\uparrow$ L_2 -norm
(could use L_1 norm)

let $v^* = \arg \max_v L(v)$

No closed form in general for

Gradient-based optimization

Basic gradient descent

Initiate v (eg $v = \vec{0}$ or random $\sim [-1, 1]^d$)

Repeat until "convergence":

for $k = 1, \dots, d$:

calculate

$$\delta_k = \frac{\partial L(v)}{\partial v_k}$$

vector δ
 $= [\delta_1, \dots, \delta_d]$
 is direction of
 steepest ascent

$$\text{let } \beta^* = \arg \max_{\beta \in \mathbb{R}} L(v + \beta \cdot \vec{\delta})$$

(usually in $(0, 1)$)

gradient descent
of $L(v): \mathbb{R}^d \rightarrow \mathbb{R}$

take some step
of some size in
dir $\vec{\delta}$
"line search"

$$v \leftarrow v + \beta^* \cdot \vec{\delta}$$

"Convergence" $\|v_{\text{new}} - v_{\text{old}}\| < \epsilon$ or $L(v_{\text{new}}) - L(v_{\text{old}}) < \epsilon$

for stopping threshold ϵ

Gradients

recall $L(v) = \sum_{i=1}^m \log p(y^{(i)} | x^{(i)}) - \frac{\lambda}{2} \|v\|^2$

$$v \cdot f(x^{(i)}, y^{(i)}) - \log \sum_{y \in \mathcal{Y}} e^{v \cdot f(x^{(i)}, y)}$$

$$\frac{\partial}{\partial v_k} \sum_{i=1}^m \text{ (red star) } = \sum_{i=1}^m f_k(x^{(i)}, y^{(i)})$$

$$\frac{\partial}{\partial v_k} \sum_{i=1}^m \text{ (blue star) } = - \sum_{i=1}^m \frac{1}{\sum_{y \in \mathcal{Y}} e^{v \cdot f(x^{(i)}, y)}} \cdot \frac{\partial}{\partial v_k} \left(\sum_{y \in \mathcal{Y}} e^{v \cdot f(x^{(i)}, y)} \right)$$

$$= - \sum_{i=1}^m \frac{\sum_{y \in \mathcal{Y}} e^{v \cdot f(x^{(i)}, y)} \cdot f_k(x^{(i)}, y)}{\sum_{y \in \mathcal{Y}} e^{v \cdot f(x^{(i)}, y)}}$$

$$\frac{\partial}{\partial v_k} v \cdot f(x^{(i)}, y)$$

$$= - \sum_{i=1}^n \sum_{y \in \mathcal{Y}} \overbrace{P_r(y | x^{(i)})}^{\text{reg}} \cdot \overbrace{f_k(x^{(i)}, y)}^{\text{reg}}$$

$$\frac{\partial}{\partial v_k} \|v\|_2 = \frac{\partial}{\partial v_k} \frac{\lambda}{2} \sum v_i^2 = \lambda v_i$$

$$\frac{\partial L(v)}{\partial v_k} = \underbrace{\sum_{i=1}^n f_k(x^{(i)}, y^{(i)})}_{\text{how many times the } k\text{-th feature } f_k \text{ is 1 in our data}} - \underbrace{\sum_{i=1}^n \sum_{y \in \mathcal{Y}} P_r(y | x^{(i)}) \cdot f_k(x^{(i)}, y)}_{\text{expected \# times feature } f_k \text{ is 1 (expectations over using current } v \text{ to generate } y\text{'s)}} - \lambda v_k$$

how many times the
k-th feature f_k is 1
in our data

expected # times feature
 f_k is 1 (expectations over
using current v to generate
 y 's)

if feature k actually occurs $\gg$ then current v 's
expected # times
it occurs

should increase $\nearrow$